

GRAND FINAL PERFORMANCE TASK: SYSTEM DEVELOPMENT & DOCUMENTATION

Project Title: OpsAI: Smart Inventory MVP (Minimum Viable Product)

Student Name: Hazel Arana

Course: BSCS - 2

Date: March 14, 2026

Table of Contents

1. Institutional Alignment, Ethics, & Social Responsibility
 2. System Requirements & Analysis
 3. Software Development Approach
 4. System Modeling
 5. Model-Driven Architecture & Architectural Design
 6. UML Design & Implementation Details
 7. Testing & Software Evolution
 8. References & Acknowledgments
-

1. Institutional Alignment, Ethics, & Social Responsibility

The **OpsAI Smart Inventory MVP** is engineered to optimize operational efficiency while reflecting core ethical principles and aligning with the institution's Vision, Guiding Morals, and Objectives (VGMO).

- **Institutional Values & Professional Ethics:** The system champions *Stewardship* and *Resourcefulness* by helping organizations accurately track inventory, preventing overpurchasing and material waste.
- **Gender and Development (GAD) & Accessibility:** The UI/UX is designed inclusively, adhering to clear typography, high-contrast colors, and intuitive layouts to ensure it is usable by individuals of all technical backgrounds and abilities.
- **Sustainable Development Goals (SDGs):** * *Goal 9 (Industry, Innovation, and Infrastructure):* Modernizing manual, paper-based tracking with a streamlined digital tool.
 - *Goal 12 (Responsible Consumption and Production):* The automated "Low Stock" and "Out of Stock" threshold logic encourages responsible supply chain management.

- **Global Citizenship Education (GCED):** By democratizing access to organized data, OpsAI fosters transparency and accountability in workplace resource management.
-

2. System Requirements & Analysis

Functional Requirements (FR)

- **FR-01 (Resource Management):** The system must allow users to input new inventory items, including Name, Category, Quantity, and a custom Low-Stock Threshold.
- **FR-02 (Automated Status Calculation):** The system must dynamically calculate and display item status (In Stock, Low Stock, Out of Stock) based on the user-defined threshold.
- **FR-03 (Data Persistence):** The system must save all inventory data to the browser's Local Storage so records remain intact after page refreshes.
- **FR-04 (Record Deletion):** The system must allow users to delete obsolete or incorrect inventory entries.

Non-Functional Requirements (NFR)

- **NFR-01 (Performance):** The system must render updates instantly (under 1 second) upon data entry.
- **NFR-02 (Usability):** The interface must be responsive and cleanly structured using a CSS Grid layout.
- **NFR-03 (Portability):** The system must run entirely in a modern web browser without requiring backend server installation.

Requirements Management

Requirements were managed using an Agile product backlog. The initial scope was focused on a Minimum Viable Product (MVP) to ensure core functional requirements (FR-01 to FR-04) were met first. This iterative approach allows for continuous validation against stakeholder needs, ensuring the system can evolve to include database integration in future iterations.

3. Software Development Approach

Software Process Model: Agile Framework

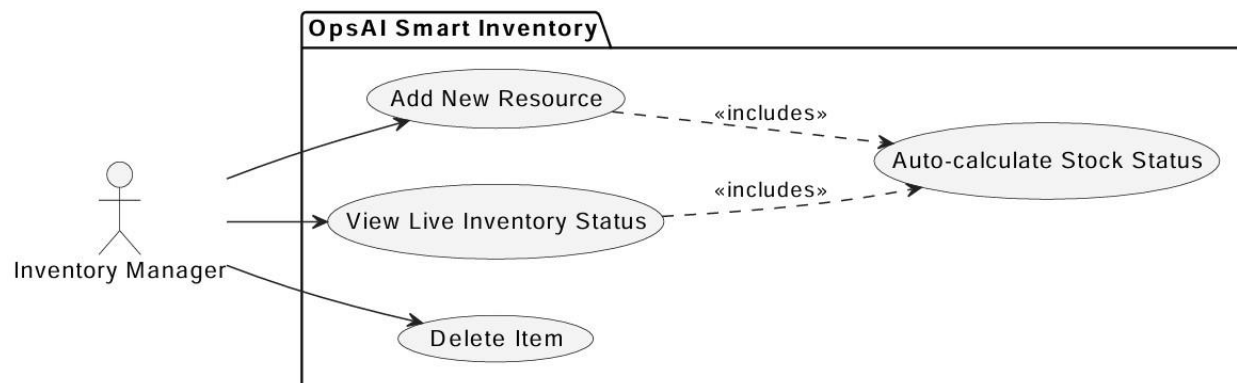
The development of OpsAI utilized an Agile methodology, prioritizing working software, iterative progress, and rapid adaptation to technical constraints.

Iteration Plan & Agile Practices

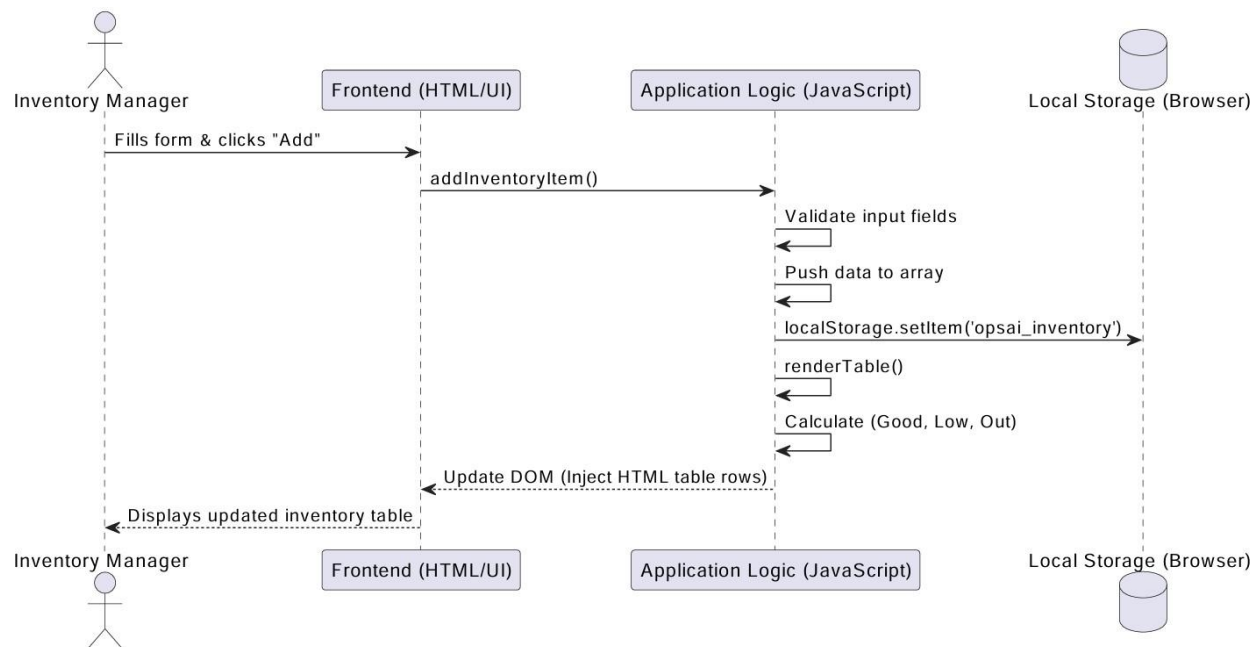
- **Sprint 1 (UI/UX Design):** Focused on creating the HTML/CSS structural layout, defining the visual theme, and ensuring a responsive CSS Grid design.
 - **Sprint 2 (Logic & Persistence):** Focused on writing the Vanilla JavaScript controllers, implementing the status calculation logic, and connecting the application to the browser's Local Storage API.
 - **User Stories:** Guided development logic. (*Example: "As an Inventory Manager, I want the system to highlight items in orange when they fall below my defined threshold, so I know when to reorder."*)
 - **Refactoring:** Code was continuously refactored during Sprint 2 to separate the DOM manipulation (`renderTable`) from the data processing (`addInventoryItem`) to maintain clean code architecture.
-

4. System Modeling

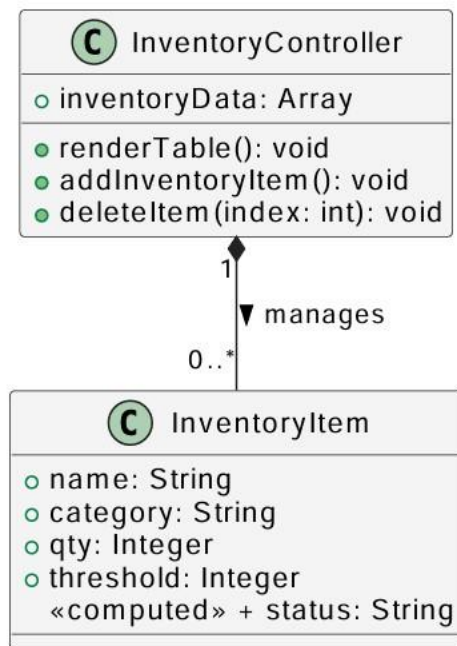
(Note: Insert the PlantUML diagrams generated from the previous code blocks here).



- **Context & Use Case Model:** [INSERT UPDATED USE CASE DIAGRAM HERE]
 - *Description:* Illustrates the Inventory Manager interacting with the core features: Adding resources, viewing live status, and deleting items, with the system handling background calculations.
- **Interaction Model (Sequence Diagram):** [INSERT UPDATED SEQUENCE DIAGRAM HERE]
 - *Description:* Details the flow of data from the User Interface (HTML), through the Application Logic (JavaScript), and into Data Persistence (Local Storage API).
- **Structural Model (Class Diagram):** [INSERT UPDATED CLASS DIAGRAM HERE]

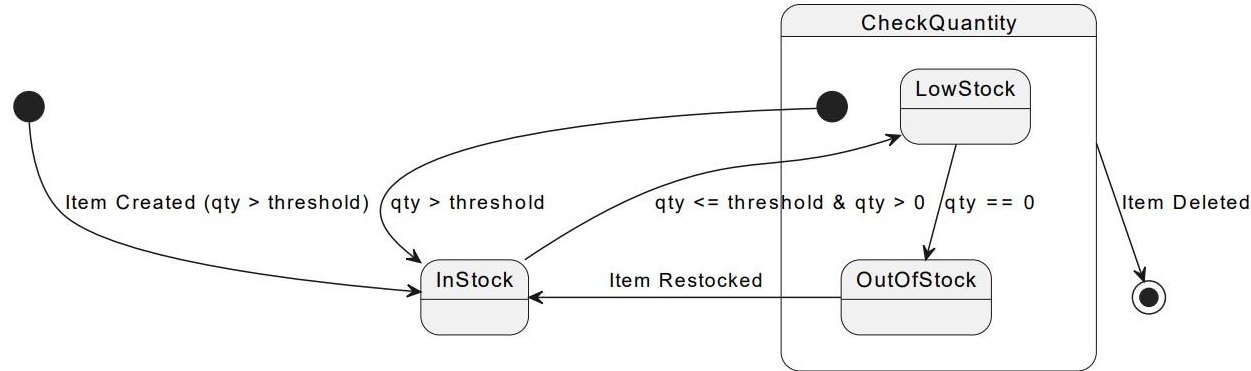


- *Description:* Shows the structure of the `InventoryItem` data object and the `InventoryController` methods that manage the application state.



- **Behavioral Model (State Diagram):** [INSERT UPDATED STATE DIAGRAM HERE]
 - *Description:* Maps the lifecycle of an inventory item as its quantity fluctuates between "In Stock", "Low Stock", and "Out of Stock".

5. Model-Driven Architecture & Architectural Design



Model-Driven Architecture (MDA)

The system was designed using a Platform-Independent Model (PIM) defining basic inventory logic, which was then translated into a Platform-Specific Model (PSM) tailored for a **Client-Side Web Application** using the Web Storage API.

Architectural Patterns

OpsAI utilizes a simplified **Client-Side MVC (Model-View-Controller) Architecture**:

- **View (Presentation Layer):** The HTML structure and Advanced CSS styling that present the dashboard to the user.
- **Controller (Application Layer):** The JavaScript functions (`addInventoryItem`, `deleteItem`) that process user input and execute business logic.
- **Model (Data Layer):** The JavaScript array (`inventoryData`) synchronized with the browser's `localStorage` to manage state. This separation ensures that the UI can be updated in the future without breaking the underlying calculation logic.

6. UML Design & Implementation Details

Implementation Choices & Tools

- **Frontend Structure:** HTML5 with semantic tags.
- **Styling:** Pure CSS3 utilizing CSS Variables (Custom Properties) for consistent theming and CSS Grid/Flexbox for responsive component alignment.
- **Application Logic:** Vanilla JavaScript (ES6), avoiding external frameworks to ensure lightweight performance and demonstrate core programming competency.

- **Data Storage:** Web Storage API (`localStorage`) for serverless, persistent client-side storage.

Working Web System


OpsAI Smart Inventory

Role: Inventory Manager

Add New Resource

Item Name

Category

Current Quantity

Low Stock Threshold

+ Add to Inventory

Live Inventory Status

Item Name	Category	Qty	Status	Action
Mac Book Pro	Electornics	1	Low Stock	Delete

7. Testing & Software Evolution

Software Testing Stages

1. **Development & Unit Testing:** Testing individual JavaScript functions in isolation. For example, verifying that entering a quantity of 0 triggers the "Out of Stock" status rather than "Low Stock".
 2. **System Testing:** Verifying that data persists accurately after entirely closing the browser and reopening the application (testing the `localStorage` implementation).
 3. **User Acceptance Testing (UAT):** Ensuring the UI form validates empty inputs correctly and provides visual feedback to the user.
- **Detected Defect & Fix Example:** During early testing, inputting string values into the Quantity field caused `NaN` (Not a Number) errors in the local storage array.
 - **Fix Applied:** Implemented the `parseInt()` function and added a validation check (`isNaN(qty)`) to explicitly reject non-numeric inputs before saving.

Software Evolution & Maintenance

As organizational needs grow, the OpsAI MVP will undergo software evolution:

- **Corrective Maintenance:** Patching any UI edge-cases discovered by users.
 - **Adaptive Maintenance:** The most significant planned evolution is migrating from clientside `localStorage` to a fully hosted backend database (e.g., MySQL via PHP) to allow multi-user synchronization.
 - **Perfective Maintenance:** Adding features like exporting the inventory table to a CSV file or adding search/filter functionality. Because the system was built using modular JavaScript functions, the cost of modifying the data layer in the future is highly minimized.
-

8. References & Acknowledgments

- Sommerville, I. (2015). *Software Engineering*. Pearson.
- Mozilla Developer Network (MDN). (n.d.). *Web Storage API Documentation*.
- [Insert your specific course materials, instructor name, or textbooks here]